Beyond Retrieval: Search Paradigms That Actually Fit Attention

What 17 experiments on a leak-free arithmetic corpus tell us about which kind of "search" belongs inside a transformer

*Working paper / research agenda. Written from measured results in this repository, not from the literature's silkiest metaphor. Numbers marked **[pre-fix]** come from before the answer-mask EOS correction (see `docs/EXPERIMENT_LOG.md`) and are inflated by ~+0.09 absolute; arm-vs-arm comparisons are unaffected.*

---

1. The honest summary of what we tested and what happened

We replaced the attention block of a shared ~99.5M-parameter trunk with seven mechanisms imported from information retrieval — learning-to-rank supervision on attention rows, inverted-index candidate generation, PRF-style query expansion, crawl-budget allocation of context, MMR-style diversification — plus controls that isolate attention entropy.

Result: **nothing beat the dense baseline per unit compute.**

arm	answer-token acc [pre-fix]	Δ vs baseline	effective attended keys (long rows)
learning-to-rank	0.4657	+0.0161 (p=.029) → **+0.0001 at equal time**	51.1
query expansion (PRF)	0.4516	+0.0020	1.59
MMR diversification	0.4501 (n=1)	+0.0005	**65.9**
baseline	0.4496	—	**1.46**
crawl budget	0.4476	−0.0020	2.02
top-k retrieval (k=32)	0.4475	−0.0021 → −0.0065 at equal time	1.53
inverted index	0.4428	−0.0068 → **−0.0130 at equal time**	**1.21**

(3 seeds per arm except MMR, still running; accuracy column is pre-fix and inflated by ~+0.09 — read the Δ column, not the level.)

The learning-to-rank arm looked like a real win until we compared at equal training time instead of equal steps, at which point the gap went to +0.0001. Its only durable effect was making attention 35× broader — and broadening is not ranking. Note that MMR diversification broadens attention **even more** (65.9 effective keys) than learning-to-rank does, and buys +0.0005 instead of +0.0161: spread alone demonstrably does not explain the LTR result either, which is exactly why "what caused it" stays open while "it was not free compute" does not.

This paper is about **why** all of it failed, because the reason is diagnostic and points somewhere specific.

---

2. Diagnosis: retrieval was the wrong species of search

Information retrieval solves a particular problem: from a **static pool of existing items**, select the few relevant to a query, where relevance is fuzzy and items are independent. Attention *can* be read that way, and that reading is what motivated the experiments. But two measurements kill it.

2.1 There is nothing to prune

Dense attention on this task mixture concentrates its mass on **~1.5 effective keys** — exp of mean row entropy, measured at eval time on every run. A trained transformer has already made the retrieval decision that all five mechanisms were trying to make better, and it made it aggressively. Top-32 selection over a 20-key support is not a hypothesis, it is an identity (we assert this in tests: at $k \geq T$, top-k attention *equals* dense attention). Worse, **76% of training rows are shorter than 32 tokens**, so for most of the data the sparse arms were provably the baseline with extra arithmetic.

The inverted-index arm is the tell. An index exists to surface relevant documents that a flat scan would miss. Instead it drove effective keys *down* from 1.46 to 1.20 and lost accuracy: restricting the candidate set on top of an already-degenerate distribution just threw away the one or two keys that mattered.

Conclusion: selection-style mechanisms are vacuous here, not weak. We were tuning a filter over a distribution that had already collapsed.

2.2 The failure is not "wrong evidence", it is "can't chain"

Compare two tasks in the same model:

task	answer length	token accuracy	greedy exact match
`TASK_CMP` (compare two numbers)	1.0 tokens	0.925	0.842
`TASK_MOD`	1.35	0.566 **[pre-fix]** / 0.128 corrected	0.051
`TASK_BINADD`	13.3	0.599	**0.000**
`TASK_DEC2BIN`	14.9	0.597	**0.000**
`TASK_MUL`	4.85	0.331	**0.000**

Single-step tasks are near the top of the table and multi-step tasks at the bottom, in a way that tracks *composition depth*, not vocabulary, context length, or how much information has to be retrieved. `TASK_BINADD` gets 60% of its individual digits right and produces the correct answer essentially never. The model can perform steps; it cannot sustain a derivation.

That is a **state-space traversal** failure, not a **database query** failure. And state spaces have a mature search literature that we had not consulted.

2.3 The enabling measurement: the model knows when it is about to be wrong

Every trajectory-search method needs a value function. We asked whether one exists for free, by scoring how well the probability the model assigns to the gold token separates its own correct from incorrect next tokens (Mann–Whitney AUC, EOS positions excluded):

```
n_correct_positions =  7,034
n_wrong_positions   = 12,798
AUC(gold-token prob discriminates correct vs wrong) = 0.928
```

For context: 0.5 is noise, 0.7 is usually enough to build on. **0.928 is a strong critic that costs nothing at inference time.** Supporting detail:

quantity	correct positions	wrong positions
mean probability on gold token	0.621	0.171
entropy of next-token distribution	0.837	1.997

And the property that matters most for search: at the **first** error in an answer the model is already as unsure (p = 0.168) as it is at later, cascaded errors (p = 0.172). Derailment announces itself *at the moment of derailment*, not only after the damage is done. A branch placed there would be well-placed.

So: the failure mode is compositional, and we have a near-perfect signal for where composition breaks. That combination is an open invitation to search — just not retrieval.

---

3. Design principles derived from the data

1. **Add compute, not selection.** Attention's distribution is already degenerate; any mechanism whose job is "choose better keys" has an empty domain. Mechanisms that spend *more work on hard tokens* change something real.
2. **Exploit verifiability.** Every one of the 27 tasks has exactly checkable ground truth (integer arithmetic, element conservation). Ranking a candidate list by likelihood is IR's core move and it caps you at the model's own accuracy. **Search that tests candidates can exceed it.** This is the single biggest untapped asymmetry in the study.
3. **Make state explicit.** Multi-digit arithmetic is a finite automaton over column position and carry. A transformer has no place to store carry except by re-reading keys it has already collapsed attention onto — which is precisely what §2.1 shows it does not do. Search over explicit states sidesteps the bottleneck instead of polishing it.
4. **Make errors recoverable.** Autoregressive decoding is a single irreversible left-to-right pass. Exact match of 0.000 with 60% per-digit accuracy means one bad digit is terminal. Any mechanism that permits *undo* attacks the dominant failure directly.
5. **Spend budget where uncertainty is.** With AUC 0.928 available, uniform compute across all answer positions is provably wasteful.

---

4. Candidate mechanisms, ranked by expected value

Each entry states the mapping, what it predicts, the cheapest experiment that could confirm it, and **what result would kill it**. The kill criteria matter more than the hopes: this study's lesson is that a mechanism can look successful for 17 runs because the confound was compute.

Tier 1 — do these first

(A) Confidence-gated decoding (branch only where you are unsure). Use the existing next-token distribution as the gate: decode greedily while probability mass is concentrated, and switch to k-way

branching with a critic when it is not. This is the cheapest possible trajectory search and it needs no training at all — the critic is the 0.928-AUC signal we already measured.

- *Prediction:* exact match rises substantially on multi-token tasks with <1.5× inference cost, because most positions need no branching (mean answer length is 5.5 tokens and most are high-confidence).
- *Minimum experiment:* inference-only sweep over the existing checkpoints. **No training run required** — hours, not days. This should literally be the next thing executed.
- *Kill criterion:* exact match gain < 2× the cost increase. Also kills it if gains appear only on tasks where a longer beam wins for trivial reasons (compare against an unpruned beam of matched average cost).

(B) Verifier-guided search — generate-and-test with an exact checker. For every task here we can decide correctness in closed form. So sample k candidates, or better, do best-first search over prefixes pruned by a verifier that checks *partial* constraints (does the digit count match? is the running remainder < divisor? are elements conserved so far?), and return the first verified answer rather than the most likely one.

- *Why this beats ranking:* likelihood-based decoding is bounded above by model accuracy; verification is not bounded that way at all. This is the asymmetry §3.2 identifies.
- *Prediction:* large exact-match gains on `TASK_MUL`, `TASK_DIV`, `TASK_MOD`, `TASK_BALANCE` — tasks where checking is dramatically easier than generating.
- *Minimum experiment:* build partial verifiers for ~6 tasks (all integer arithmetic), run sample-and-filter at $k \in \{1, 4, 16, 64\}$ on existing checkpoints. Reports both accuracy **and** mean candidates consumed, so the compute cost is never hidden.
- *Kill criterion:* if accepted answers at k=16 are no better than top-1 greedy at equal expected cost, verification adds nothing (which would itself be surprising and worth knowing).

(C) Neural A* / best-first over explicit arithmetic state. This is the A* idea, made concrete rather than metaphorical. For column-wise arithmetic the true state is small and well-defined: **(column index, carry)**. Define

- state `s = (position i, carry c)`, successor states from learned per-column transition probabilities taken from the transformer's own digit distribution;
- `g(s)` = accumulated negative log-likelihood of the prefix (exactly what makes it A* rather than greedy);
- `h(s)` = critic-derived estimate of remaining cost, ideally **admissible by construction** — a lower bound obtained from the verifier on the unsolved suffix;
- closed set over canonical `(i, c)` pairs, so different token prefixes reaching the same arithmetic state are recognised as duplicates.

The reason this is not just beam search with extra steps: duplicate detection means work is never repeated, and an admissible `h` means the first goal popped from the open list is optimal — properties neither greedy nor beam decoding has.

- *Prediction:* on multi-digit addition/subtraction, where carry is provably the missing state, this should close a large fraction of the gap between ~35% per-digit accuracy and near-zero exact match. `TASK_SUB` (0.268 token acc, 4.16-token answers) and `TASK_ADD` are the sharpest tests.
- *Minimum experiment:* implement carry-state A* on top of an existing checkpoint for `TASK_ADD`/`TASK_SUB` only, with `h = 0` (Dijkstra) as the ablation that isolates the heuristic's contribution from duplicate detection's.

- *Kill criterion:* if `h = 0` performs the same as the full heuristic, there is no learned signal to exploit and this collapses to (A). If A* with a perfect hand-written `h` still fails, then carry is not the missing ingredient and the diagnosis in §2.2 needs revision.

Tier 2 — promising, more expensive

(D) AND/OR (AO*) decomposition with verifiable subgoals. Multiplication of two multi-digit numbers decomposes into partial products that can each be checked independently: an OR-node (several ways to solve) over AND-nodes (all subparts must hold). AO* handles exactly this cost-decomposable structure and expands whichever subproblem has the best total estimated cost. This also gives a natural curriculum — subgoals are individually verifiable, so partial credit exists, which plain autoregressive decoding lacks.

- *Prediction:* gains concentrated on `TASK_MUL`, `TASK_HEXMUL`, `TASK_CHEM_SUM` (sum of several molar masses) rather than on single-pass tasks.
- *Kill criterion:* if per-subgoal verification plus recombination does not beat end-to-end generation at equal cost, decomposition is not buying structure.

(E) Constraint propagation and conflict-directed backjumping. Carry consistency in arithmetic behaves like unit propagation in SAT: a wrong digit creates a local contradiction that could be detected immediately, and CDCL-style search can then jump back past the *decision responsible* rather than retrying the most recent token. This is the mechanism that directly attacks "one bad digit is terminal" (§3.4), and it is a much better structural match for these tasks than anything in IR.

- *Prediction:* on long-answer tasks (`TASK_BINADD`, 13.3 tokens; `TASK_DEC2BIN`, 14.9), backjumping should beat simple resampling-by-rejection at equal cost, because it reuses the correct prefix instead of discarding everything.
- *Kill criterion:* if backjumping $\approx$ resampling from scratch at matched compute, the contradiction signal is too weak or too late to localise blame.

(F) Bidirectional / meet-in-the-middle search. Division and modular arithmetic have cheap reverse checks: guess a quotient, multiply back, compare remainders. Searching forward and backward and meeting in the middle shrinks the effective space exponentially for such tasks.

- *Prediction:* task-specific but large on `TASK_DIV`, `TASK_MOD`, `TASK_HEXDIV`.
- *Kill criterion:* gains appear only where the reverse check is near-bijective (i.e. the win is arithmetic trickery, not search).

Tier 3 — worth a thought experiment before any GPU time

(G) Anytime budget allocation as the generalisation of "crawl budget". Our crawl-budget arm failed, but the idea underneath it — spend a limited resource where marginal value is highest — survived in §3.5 and is what makes (A) work. Reframed as *per-token compute allocation* rather than *context selection*, it becomes adaptive-depth computation: some tokens need one forward pass, a few need ten. This is the version of the IR idea that we should have tried.

(H) MCMC / Metropolis over latent derivations. Sample whole derivations, accept/reject by a scoring rule including verifier feedback. Attractive because it needs no admissible heuristic and parallelises; unattractive because mixing on discrete sequences is slow and our answers are short enough that exact search is cheaper.

(I) Successive halving over candidate prefixes. A bandit-flavoured way to spend a fixed inference budget: many candidates at shallow depth, progressively narrowed. Mostly a smarter implementation of (A)/(B), worth stealing pieces from rather than adopting wholesale.

---

5. Why the IR concepts failed, mapped to their replacements

imported IR concept	why it was vacuous here	search-theoretic replacement
relevance ranking / learning-to-rank	attention already at 1.5 keys — no ranking left to do	rank *candidate derivations*, not keys (A, B)
inverted index / posting lists	candidate pool is tiny and already collapsed	explicit state space with duplicate detection (C)
query expansion (PRF)	there is no query ambiguity; the prompt is exact	subgoal decomposition generates new "queries" legitimately (D)
crawl budget over context	nothing to exclude from a 15-token context	per-token compute budget (A, G)
MMR diversification	redundant keys are already suppressed	sample diverse *derivations*, verify each (B, H)
static corpus of documents	answers do not exist until derived	online search through states with backtracking (C, E)

The pattern: every IR concept assumes **a fixed body of existing items to select among**. Arithmetic has no such body. The moment we treated the answer as something *constructed* rather than *found*, the useful ideas came from A*, AO*, and SAT — not from search engines.

---

6. Methodological corrections this agenda depends on

Stated because they change how results must be read, and because an agenda built on inflated numbers is decoration.

- **Metric bug found during this work.** The dataset appends an end-of-sequence token to every row, and the answer mask counted it as a scored answer position — giving every row one free, ~95%-predictable hit. Measured on the baseline: accuracy 0.952 on those 15.4% of positions vs **0.358** on real answer tokens, inflating the headline by **+0.092 absolute**. Distortion is task-dependent (largest for short answers), so per-task rankings and every "lift over trivial floor" figure were overstated. Arm-vs-arm comparisons hold; absolute claims do not. Fixing before any of §4 runs.
- **Equal steps is not equal compute.** The learning-to-rank arm's +0.016 became +0.0001 at matched wall-clock. Every mechanism in §4 must report accuracy against **cost** (candidates consumed, forward passes, seconds), not just accuracy.
- **Three tasks cannot be scored per-task.** `TASK_BALANCE`, `TASK_ION`, `TASK_POW` have 7, 9 and 132 held-out answer tokens. They are excluded from per-task claims; they are 0.09% of test answer tokens so aggregates stand.
- **Exact-match per-task samples are small** (≈ 12 per task×difficulty cell). The derailment ratios computed from them are indicative, not settled; the *gap* between single-token and multi-token tasks is robust, its per-task decomposition is not.

6.5 Should the models be smaller (≈10M) to iterate faster?

Reasonable instinct, and partly right — but it trades away exactly what this study's conclusions rest on, so it needs a condition attached.

What shrinking buys. At the current shape, measured step times at bs=32 over 21 runs:

arm	ms/step	vs baseline
baseline (99.5M)	41.6	—
top-k / inverted index	45.2 / 45.7	+9%
learning-to-rank	50.7	+22%
MMR diversification	52.2	+25%
query expansion	70.0	+68%
crawl budget	81.3	+96%

A d=384/L=6 trunk is 10.8M parameters — **9.2× fewer FLOPs per step** (332 vs 3,057 GFLOP/step estimated at 6·N·T·B). Realistically expect **3–5× wall-clock**, not 9×, because kernel-launch and dataloader costs are fixed: the pipeline here is already optimised to ~40 ms/step for a 99.5M model, which means a substantial floor is *not* the matmul. (A direct measurement could not be taken while the grid holds all 8 GB; it is queued.)

What shrinking costs. Two of this paper's load-bearing findings are empirical claims about a *specific operating point*, and neither is guaranteed to survive:

1. **Attention collapses to ~1.5 effective keys.** A smaller model has less capacity to sharpen with, so 10M may show broader attention — and then "pruning is vacuous" stops being true for reasons that have nothing to do with the mechanism under test. The sparse arms could look *better* at 10M purely because there is more to prune.
2. **The critic signal (AUC 0.928) makes trajectory search viable.** That is a property of how confidently the trained model tracks its own errors. A capacity-starved model may be confidently wrong, and then §4(A)–(C) all lose their precondition at once — while still appearing to fail "for interesting reasons."

There is also a third-order risk specific to this corpus: several tasks are already hard at 99.5M (`TASK_MUL` exact match 0.000). At 10M, more cells fall to zero, and **a metric that saturates at zero cannot distinguish mechanisms.** Search experiments need headroom; shrinking eats headroom fastest on exactly the multi-step tasks §2.2 uses as evidence.

Recommendation: two tiers, with a replication gate.

- **Tier S (10M) for screening only.** Cheap ranking of which candidate mechanisms deserve real runs — especially §4(A) confidence-gated decoding and §4(B) verifier-guided search, both of which are *inference-time* methods that need no training at all.
- **Gate before trusting Tier S:** the diagnostics must replicate. Concretely, a 10M model is usable as a proxy only if it reproduces (i) effective keys in the same regime, and
    1. critic AUC ≥ ~0.85. If either fails, Tier S results do not transfer and only create plausible-looking negatives — the most expensive kind of cheap result.
- **Tier L (99.5M) for any claim we intend to publish**, since that is where the reported behaviour was observed.

The faster win is not shrinking. The immediate cost in this study is not model size, it is that we keep *training* to answer questions that need only evaluation:

what we would otherwise do	what actually suffices
retrain 21 runs to fix the scoring bug	**rescore saved checkpoints** — weights don't depend on the metric mask (`src/rescore.py`)
train arms to test trajectory search	inference-only sweeps on existing weights (§4 A, B)
train at 10M to iterate faster	reuse weights + measure speedup once, then decide

So: yes to a 10M screening tier with the replication gate, but the first thing worth doing with freed GPU time is rescoring and inference-only search experiments — both cheaper than any model-size reduction and neither constrained by it.

---

7. What I would stop doing

- Any further selection-over-keys mechanism at this data scale. §2.1 forecloses it; the sweep over k $\in \{1,2,4,8\}$ can confirm the boundary cheaply and then close the question.
- Hyperparameter sweeps on arms that are $\leq$ baseline at matched compute (inverted index is −0.013). Tuning a negative does not make it positive.
- Reporting accuracy without stating whether EOS was masked, and without cost.

---

8. Suggested next actions, in order

1. **Fix the answer mask** so no scored position targets EOS; add a permanent assertion. Everything else is interpretation-dependent until then.
2. **Run (A) confidence-gated decoding as inference-only** on existing checkpoints — no training needed, and it tests the central claim (that the 0.928-AUC critic is usable) within hours.
3. **Build partial verifiers for ~6 integer tasks** and run (B) sample-and-filter with cost accounting. Highest expected ceiling in the whole agenda because it escapes the likelihood ceiling.
4. **Prototype carry-state A* on `TASK_ADD`/`TASK_SUB`** with the `h = 0` ablation — this is the version of "A* inside a transformer" that our own data actually supports, and its failure would be informative rather than merely expensive.

The through-line: **the model already knows how to look; it does not know how to recover.** Search over derivations — with an explicit state, a critic we have already measured, and a verifier that costs nothing — targets the second problem. Retrieval only ever addressed the first, which is why seven variants of it left the numbers where they started.

---

9. What happened when §4 was implemented

§4 was written as predictions. This section reports measurements against them, from `src/verifiers.py`, `src/search.py`, `src/headroom.py`, `src/eval_search.py`, `src/screen.py` and `analysis/search_aggregate.json`. Where a prediction was wrong it is marked as wrong rather than quietly dropped.

9.1 (B) Verifier-guided search: the central contrast, at matched compute

Tier-L baseline checkpoint(s), stratified test rows, 16 samples per rollout. Pooled over 3 seeds (`analysis/search_aggregate.json`):

method	exact match	vs greedy	cost ×
best_of_n_verif	**0.1120** ±0.0075	+0.0680	16.0
self_consistency (V+vote)	0.1111 ±0.0098	+0.0672	16.0
gated_pure (confidence-gated)	0.0879 ±0.0083	+0.0439	8.65
verif_greedy (constrained decoding)	0.0525 ±0.0023	+0.0086	**1.00**
best_of_n (likelihood only)	0.0508 ±0.0034	+0.0069	16.0
vote_only (no verifier)	0.0474 ±0.0034	+0.0034	16.0
greedy	0.0439 ±0.0030	—	1.00

The prediction in §3 held and is now isolated. `best_of_n` and `best_of_n_verif` draw identical samples at identical cost and differ in one bit of information — whether an exact checker is consulted. Verifier value: **+0.0612 absolute**. Likelihood-only re-ranking of the same samples recovers +0.0069, i.e. ~11% of what verification achieves, which is the concrete form of "re-ranking cannot promote an answer the model ranks below a wrong one".

The control that makes this interpretable is `vote_only`: same 16 samples, majority vote, **no verifier at all** → +0.0034 (Tier S: −0.0026). Extra compute alone buys essentially nothing; the gain is information the model does not possess. This is the result §4(B) needed and did not have when it was written as a prediction.

`verif_greedy` is the practically interesting row: +0.0086 at **1.00× cost**. It is the only mechanism in the entire study — attention arms included — that improves accuracy without paying for it, because constrained decoding changes *which* token is chosen rather than adding rollouts.

9.2 Where verification actually bites: format, not cleverness

Measured fraction of the vocabulary still admissible after one answer token (i.e. pruning power at step 1): `TASK_CMP` **0.008** — a three-way choice, maximally constrained — versus **0.119** for free-length integer arithmetic, which is exactly the ten digits, i.e. **no pruning whatsoever**.

Headroom analysis (`src/headroom.py`, 24 samples/problem) sharpens this into a per-task statement: ceiling rises 0.0599 → 0.1521 from 1→24 samples with ~21 distinct answers per problem, but the headroom is concentrated in format-constrained tasks (`TASK_MOD` +0.750, `TASK_ADD` +0.333, `TASK_CMP` +0.167) and is **exactly zero on 9 of 19 tasks**, where no sample out of 24 ever verifies. On those tasks no selection policy — best-first, MCTS, A*, arbitrary oracle over samples — can help, because the correct answer is never generated.

Consequence for §4(B), stated as a correction: verification's value is not uniform across the suite and is bounded above by generation coverage. The claim "checking is easier than generating" is true of the *checker* but irrelevant if the generator never proposes the thing to be checked. Any future search work here should report per-task coverage first; a single aggregate accuracy number over this corpus mixes

tasks where search is powerful with tasks where it is provably inert, and averages them into something uninterpretable.

9.3 (A) Confidence-gated decoding: prediction half-right

§4(A) predicted "exact match rises substantially at <1.5× inference cost". Measured: `gated_pure` gains +0.0439 but spends **8.65**×. The gate signal itself is real and strong — critic AUC 0.928 at Tier L, 0.939–0.964 at Tier S — but the *cost* prediction was wrong by roughly 6×. Reason: mean per-token logprob of a 5.5-token answer is low on most arithmetic problems, so the gate escalates far more often than "only where unsure" suggests (209 of 387 rows). A better gate would be entropy-per-position or an explicit abstain head, not a threshold on average logprob; §4(A)'s mechanism is sound, its cost model was not.

The oracle-leakage check worked as intended: `gated` (which may ask the verifier whether greedy already succeeded) gains +0.0672 at 13.4×, while `gated_pure` (confidence only, no oracle stop decision) gains +0.0439 at 8.65×. Reporting both put the size of that optimism on the record: ~0.023 of gated's gain came from peeking.

9.4 (C) Carry-state A*: abandoned as vacuous, and why that is a finding

§4(C) proposed A* over `(column index, carry)` with model-derived transition probabilities. Implementing it exposed a fatal circularity: for fixed `a op b`, given `(column i, carry_in)` the output digit is **determined by arithmetic**, so enumerating "successor states" computes the answer without consulting the model at all. Such a method would report ~100% accuracy and credit it to search. It was deleted (see commit history for `src/search.py`) rather than reported as a success, because a verifier or transition function that *is* the solution measures nothing about the model.

A non-vacuous version must draw its transitions from the model while drawing only *constraint checking* from arithmetic — which is precisely `verif_greedy` in §9.1, already measured at 1.00× cost. So (C) does not survive as a separate mechanism; it collapses into (B). The kill criterion in §4(C) anticipated this ("if A* with a perfect hand-written h still fails, carry is not the missing ingredient"); the actual outcome is stronger and less interesting — the hand-written h is the whole answer, so there is nothing left to test.

9.5 Tier S (10M) replication gate: FAILED for its intended purpose

Trunk d=384/L6/H6/ffn×4 = 10.8M params, 12 ms/step vs 41.6 ms at 99.5M — an ~3.5× per-step saving, so the tier is cheap exactly as §6.5 assumed. Gate results:

gate	criterion	result
1	effective keys in Tier-L regime (0.2–5× of 1.46)	**FAIL** — Tier-S baseline 7.8–12.0
2	critic AUC ≥ 0.85	PASS — 0.939–0.964
3	non-zero multi-step EM headroom	PASS — 0.017–0.034 (Tier L: 0.0088)

§6.5's own hazard was realised: the small model spreads attention over ~6–8× more effective keys, so pruning arms would look better at Tier S purely because there is more to prune, and diversification arms would look worse. **Tier S must not be used to screen attention mechanisms.** It remains usable for inference-time search (gate 2 passes strongly, and search methods do not depend on the attention-spread regime) — which is how it was used here, and why the §9.1 numbers were replicated at both tiers (+0.061 Tier L, +0.074 Tier S).

An unplanned result worth more than the screening: at equal data exposure the 10.8M model matches or beats the 99.57M trunk (test token accuracy 0.361 vs 0.357; multi-step EM 2–4× higher). Trunk capacity was never the binding constraint on this corpus, which retroactively justifies §6.5's suspicion that model size was not where the study's cost should go.

9.6 Methodological notes from implementing §4

- **Null results from broken wiring are the dominant risk.** Two separate bugs made the verifier silently do nothing (a missing `vocab` argument swallowed by an "undecidable" exception guard; a trailing `<eos>` making exact match 0.00 on every row). Both presented as *evidence that search does not work*. Unit tests of verifier logic passed throughout. The fix that matters is `tests/test_search_wiring.py`, which asserts the seam: no un-vocab'd verifier calls, no EOS leaking into answers, and selection must equal its oracle.
- **A verifier must be tested against the generator, not against intuition.** 19 tasks × ground-truth-must-verify, perturbed-must-reject, and *no true prefix may ever be pruned*. Three of four verifier bugs were caught by the prefix test, which is the one nobody writes unless they have been burned.
- **Chemistry semantics are imported from `src/tasks.py`, never copied.** A duplicated atomic weight table would silently disagree with the corpus the moment chemistry changed.

9.7 Promotion to every attention arm: the gains replicate, the arm ranking does not persist

Both promoted mechanisms were then measured on all seven non-baseline arms at 99.5M (`tools/promote_search.sh`, `analysis/search_aggregate.json` → per_arm). Since these are inference-time methods sharing weights with the arm they are applied to, "promoting to full scale" required no training — only coverage, which had been missing: everything before this was measured on baseline checkpoints alone.

arm	greedy EM	verif_greedy (1.0×)	best_of_n_verif (16×)	gain
crawl_budget	0.0646	0.0672	0.1189	+0.0543
topk_sparse	0.0594	0.0620	0.1214	+0.0620
mmr_diversify	0.0491	0.0594	0.1163	+0.0672
query_expand	0.0491	0.0543	0.1111	+0.0620
ltr_rank	0.0465	0.0491	0.1266	+0.0801
uniform_mix	0.0439	0.0491	0.1292	+0.0853
posting_index	0.0388	0.0491	0.1111	+0.0724

Three findings:

1. **Replication.** Verification helps on 7/7 arms (+0.054 to +0.085). The effect is not an artifact of the checkpoint it was discovered on.
2. **Verification compresses arm differences.** Under greedy decoding the arms span 0.0388–0.0646 (1.67×); after verifier selection they span 0.1111–0.1292 (1.16×). The attention mechanism that took 21 training runs to separate is worth less than the decoding policy applied afterwards. This is the study's sharpest statement of its main thesis: the search was in the wrong place.
3. **Gains are not ordered by greedy strength.** The largest gains go to `uniform_mix` (+0.0853) and `ltr_rank` (+0.0801) — the two arms with the broadest attention — while the weakest greedy arm

(`posting_index`) gains +0.0724. A plausible mechanism, stated as a hypothesis rather than a result: broader attention yields more *diverse* samples, and verifier selection can only exploit diversity it is given. That is consistent with §9.2's coverage point and testable by measuring per-arm sample diversity, which has not been done.

The Spearman correlation between per-arm greedy rank and post-verification rank is −0.01, but with 7 arms the Fisher standard error is ~0.50, so this does **not** support "rankings do not transfer" — it supports "we cannot resolve the relationship with 7 arms", which is what the report says. A draft sentence here claimed posting_index gained the most; checking the table showed that was false, and the sentence is now generated from the data instead of from memory.

9.8 Equal-compute verdict on the attention arms, recomputed after the metric fix

§4C of the report now carries both comparisons on corrected metrics: `uniform_mix` reaches 45.2 effective keys (~30× baseline's 1.5) and scores +0.0005 (z=0.2), while `ltr_rank` at 51.1 effective keys scores +0.0169 (z=6.7). At matched wall-clock ltr_rank's margin is **−0.0004**. So neither "attention got broader" (killed by the spread control) nor "it is a better use of compute" (killed by equal time) survives; `posting_index` is the only arm significantly different from baseline in either direction, and it is worse (−0.0070, z=−2.5).

This column was previously wrong in an instructive way: the budget arithmetic mixed milliseconds with steps, so both arms were evaluated at full length and ltr_rank appeared to win equal-time by +0.0171 — the opposite sign of the truth. `tests/test_equal_compute.py` now pins that arithmetic with a fixture reproducing the real pattern (+0.0169 equal-steps, +0.0000 equal-time).